

Assignment:

1. Get employee details whose salary is the highest in the city mumbai.

```
SELECT *
FROM EMPS
WHERE LID = (
    SELECT LID
    FROM LOCATIONS
    WHERE CITY = 'MUMBAI'
)
ORDER BY SAL DESC
LIMIT 1;
```

2. Find employees who handled only Delivered orders.

```
SELECT *
FROM EMPS
WHERE EID IN (
    SELECT EID
    FROM ORDERS
    WHERE STATUS = 'Delivered');
```

3. Find employees who handled both Delivered and Cancelled orders.

```
SELECT *
FROM EMPS
WHERE EID IN (
    SELECT EID
    FROM ORDERS
    WHERE STATUS = 'Delivered')
AND EID IN (
    SELECT EID
    FROM ORDERS
    WHERE STATUS = 'Cancelled');
```

4. Get employee names who handled more orders than the average number of orders handled by employees.

```
SELECT FNAME
FROM EMPS
WHERE EID IN (
    SELECT EID
    FROM ORDERS
    GROUP BY EID
    HAVING COUNT(*) > (
        SELECT AVG(order_count)
        FROM (
            SELECT COUNT(*) AS order_count
```

```

        FROM ORDERS
        GROUP BY EID
    ) AS X ));

```

5. Find employees whose commission is greater than the average commission of all employees.

```

SELECT *
FROM EMPS
WHERE COMM > (
    SELECT AVG(COMM)
    FROM EMPS
);

```

6. List employees whose DOB is earlier than all employees in the DELIVERY job.

```

SELECT *
FROM EMPS
WHERE DOB < (
    SELECT MIN(DOB)
    FROM EMPS
    WHERE JOB = 'DELIVERY');

```

7. Find customers who have placed orders from restaurants located in the same city as the customer.

```

SELECT NAME
FROM CUSTOMERS
WHERE ORDER_ID IN (
    SELECT O.ORDER_ID
    FROM ORDERS O, RESTAURANTS R
    WHERE O.RESTAURANT_ID = R.RESTAURANT_ID
    AND R.CITY = (
        SELECT CITY
        FROM LOCATIONS L
        WHERE L.LID = CUSTOMERS.LID )
);

```

8. Find customers who never placed any order.

```

SELECT *
FROM CUSTOMERS
WHERE CID NOT IN (
    SELECT CID
    FROM CUSTOMERS
    WHERE ORDER_ID IS NOT NULL
);

```

9. Get customers whose order amount is higher than the average payment amount.

```
SELECT *
FROM CUSTOMERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM PAYMENTS
    WHERE AMOUNT > (
        SELECT AVG(AMOUNT)
        FROM PAYMENTS )
);
```

10. Find customers who placed orders but payment status is Pending or Failed.

```
SELECT *
FROM CUSTOMERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM PAYMENTS
    WHERE STATUS IN ('Pending', 'Failed')
);
```

11. List customers who placed more than one order.

```
SELECT *
FROM CUSTOMERS
WHERE CID IN (
    SELECT CID
    FROM ORDERS
    GROUP BY CID
    HAVING COUNT(*) > 1
);
```

12. Find restaurants whose average rating is higher than the overall average rating.

```
SELECT NAME
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM REVIEWS
    GROUP BY RESTAURANT_ID
    HAVING AVG(RATING) > (
        SELECT AVG(RATING)
        FROM REVIEWS
    )
);
```

13. List restaurants that never received any review.

```

SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID NOT IN (
    SELECT RESTAURANT_ID
    FROM REVIEWS
);

```

14. Find restaurants where total order amount is maximum.

```

SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM MENU_ITEMS
    GROUP BY RESTAURANT_ID
    HAVING SUM(PRICE) = (
        SELECT MAX(TOTAL)
        FROM (
            SELECT SUM(PRICE) AS TOTAL
            FROM MENU_ITEMS
            GROUP BY RESTAURANT_ID
        ) X )
);

```

15. Get restaurants that have menu items priced higher than the average menu price.

```

SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM MENU_ITEMS
    WHERE PRICE > (
        SELECT AVG(PRICE)
        FROM MENU_ITEMS )
);

```

16. Find restaurants that received only 5-star ratings.

```

SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM REVIEWS
    GROUP BY RESTAURANT_ID
    HAVING MIN(RATING) = 5
    AND MAX(RATING) = 5
);

```

17. Find orders whose payment amount is greater than the average payment amount.

```

SELECT *
FROM ORDERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM PAYMENTS
    WHERE AMOUNT > (
        SELECT AVG(AMOUNT)
        FROM PAYMENTS
    )
);

```

18. Display orders that are Delivered but payment is not Completed.

```

SELECT *
FROM ORDERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM PAYMENTS
    WHERE STATUS != 'Completed'
)
AND STATUS = 'Delivered';

```

19. Find orders that contain more than one menu item.

```

SELECT *
FROM ORDERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM MENU_ITEMS
    GROUP BY ORDER_ID
    HAVING COUNT(*) > 1
);

```

20. Get orders where order date is later than all orders handled by employee 2.

```

SELECT *
FROM ORDERS
WHERE ORDER_DATE > (
    SELECT MAX(ORDER_DATE)
    FROM ORDERS
    WHERE EID = 2
);

```

21. Find orders whose total menu item price is higher than the average order value.

```

SELECT *
FROM ORDERS

```

```

WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM MENU_ITEMS
    GROUP BY ORDER_ID
    HAVING SUM(PRICE) > (
        SELECT AVG(TOTAL)
        FROM (
            SELECT SUM(PRICE) AS TOTAL
            FROM MENU_ITEMS
            GROUP BY ORDER_ID
        ) X )
);

```

22. Find locations where both employees and customers exist.

```

SELECT *
FROM LOCATIONS
WHERE LID IN (
    SELECT LID
    FROM EMPS
)
AND LID IN (
    SELECT LID
    FROM CUSTOMERS
);

```

23. List locations where number of customers is greater than number of employees.

```

SELECT *
FROM LOCATIONS
WHERE LID IN (
    SELECT C.LID
    FROM CUSTOMERS C
    GROUP BY C.LID
    HAVING COUNT(*) > (
        SELECT COUNT(*)
        FROM EMPS E
        WHERE E.LID = C.LID
    )
);

```

24. Find locations where no orders were delivered.

```

SELECT *
FROM LOCATIONS
WHERE LID NOT IN (
    SELECT C.LID
    FROM CUSTOMERS C
    WHERE C.ORDER_ID IN (

```

```

        SELECT ORDER_ID
        FROM ORDERS
        WHERE STATUS = 'Delivered'
    )
);

```

25. Find employees who handled orders from all restaurants.

```

SELECT *
FROM EMPS
WHERE EID IN (
    SELECT EID
    FROM ORDERS
    GROUP BY EID
    HAVING COUNT(DISTINCT RESTAURANT_ID) = (
        SELECT COUNT(*)
        FROM RESTAURANTS
    )
);

```

26. Find customers who ordered from the same restaurant as customer 'KIRAN RAJ'.

```

SELECT *
FROM CUSTOMERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM ORDERS
    WHERE RESTAURANT_ID = (
        SELECT RESTAURANT_ID
        FROM ORDERS
        WHERE ORDER_ID = (
            SELECT ORDER_ID
            FROM CUSTOMERS
            WHERE NAME = 'KIRAN RAJ'
        )
    )
);

```

27. Find employees who handled orders with payment method = UPI more than average.

```

SELECT *
FROM EMPS
WHERE EID IN (
    SELECT EID
    FROM ORDERS
    WHERE ORDER_ID IN (
        SELECT ORDER_ID
        FROM PAYMENTS
        WHERE METHOD = 'UPI'
    )
);

```

```

GROUP BY EID
HAVING COUNT(*) > (
    SELECT AVG(CNT)
    FROM (
        SELECT COUNT(*) AS CNT
        FROM ORDERS
        WHERE ORDER_ID IN (
            SELECT ORDER_ID
            FROM PAYMENTS
            WHERE METHOD = 'UPI'
        )
    )
    GROUP BY EID
) X
);

```

28. Find restaurants whose orders were handled by more than one employee.

```

SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM ORDERS
    GROUP BY RESTAURANT_ID
    HAVING COUNT(DISTINCT EID) > 1
);

```

29. Get customers whose city has the highest number of orders.

```

SELECT *
FROM CUSTOMERS
WHERE LID IN (
    SELECT LID
    FROM CUSTOMERS
    WHERE ORDER_ID IS NOT NULL
    GROUP BY LID
    HAVING COUNT(*) = (
        SELECT MAX(CNT)
        FROM (
            SELECT COUNT(*) AS CNT
            FROM CUSTOMERS
            WHERE ORDER_ID IS NOT NULL
            GROUP BY LID
        ) X
    )
);

```

30. Find employees whose salary is in the top 3 salaries using subquery.

```

SELECT *
FROM EMPS E1
WHERE 3 > (

```



```

SELECT COUNT(DISTINCT E2.SAL)
FROM EMPS E2
WHERE E2.SAL > E1.SAL
);

```

31. Find restaurants that have menu items but no orders.

```

SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM MENU_ITEMS
)
AND RESTAURANT_ID NOT IN (
    SELECT RESTAURANT_ID
    FROM ORDERS
);

```

32. Find customers who reviewed a restaurant but never ordered from it.

```

SELECT *
FROM CUSTOMERS
WHERE CID IN (
    SELECT CID
    FROM REVIEWS
    WHERE RESTAURANT_ID <> (
        SELECT RESTAURANT_ID
        FROM ORDERS
        WHERE ORDER_ID = (
            SELECT ORDER_ID
            FROM CUSTOMERS C
            WHERE C.CID = REVIEWS.CID
        )
    )
);

```

33. Find employees whose job role salary average is greater than overall employee average salary.

```

SELECT *
FROM EMPS E
WHERE JOB IN (
    SELECT JOB
    FROM EMPS
    GROUP BY JOB
    HAVING AVG(SAL) > (
        SELECT AVG(SAL)
        FROM EMPS
    )
);

```

34. Find restaurants where total revenue is less than the average restaurant revenue.

```
SELECT *
FROM RESTAURANTS
WHERE RESTAURANT_ID IN (
    SELECT RESTAURANT_ID
    FROM MENU_ITEMS
    GROUP BY RESTAURANT_ID
    HAVING SUM(PRICE) < (
        SELECT AVG(TOTAL)
        FROM (
            SELECT SUM(PRICE) AS TOTAL
            FROM MENU_ITEMS
            GROUP BY RESTAURANT_ID
        ) X
    )
);
```

35. Get orders where payment amount equals the maximum payment made by any customer.

```
SELECT *
FROM ORDERS
WHERE ORDER_ID IN (
    SELECT ORDER_ID
    FROM PAYMENTS
    WHERE AMOUNT = (
        SELECT MAX(AMOUNT)
        FROM PAYMENTS
    )
);
```

36. Find customers who placed orders in multiple cities.

```
SELECT *
FROM CUSTOMERS
WHERE CID IN (
    SELECT CID
    FROM ORDERS O, RESTAURANTS R
    WHERE O.RESTAURANT_ID = R.RESTAURANT_ID
    GROUP BY CID
    HAVING COUNT(DISTINCT R.CITY) > 1
);
```